FHEM Code für Diplomarbeit

Ich schreibe eine Diplomarbeit über Smart Home Systeme. Ich habe mich für FHEM entschieden und muss nun mit FHEM bzw. Perl einen Code schreiben er die ganzen Funktionen einbinden muss. Wir haben die Funktionen(Licht Steuerung, Heizung zu regulieren oder auf einen gewissen Grad Celsius einstellen, Einen Temperatur Messer bzw. die ganze Wetterstation). Ich muss auf meine ganze Theorie dazu schreiben gib mir mal den Code für diese Funktionalität und die Theorie damit man diesen Code schreiben Kann

Unten bekommst du **(1) die Theorie**, die du in der Diplomarbeit verwenden kannst (so, dass man versteht *warum* der Code so aussieht), und **(2) ein vollständiges Beispiel-„Grundgerüst" für FHEM** inkl. **Licht**, **Heizung (Solltemperatur)** und **Wetter/Temperatur (Wetterstation + Forecast)**.
Das Ganze ist bewusst **hardware-neutral** gehalten (weil es in FHEM je nach Funkstandard/Bus/Bridge unterschiedlich ist). Du musst dann nur die **Geräte-Module** (z. B. MQTT, KNX, Homematic, Shelly, Zigbee2MQTT …) austauschen, aber **Logik und Struktur** bleiben gleich.

---

1) Theorie (für die Diplomarbeit)

1.1 Grundprinzip von FHEM

FHEM ist ein serverbasiertes Smart-Home-System, das Geräte über unterschiedliche Protokolle/Standards einbindet und deren Zustände zentral verwaltet. Die Konfiguration erfolgt in der Regel über **Textkommandos** (Konfigurationsdatei `fhem.cfg`) oder über das Web-Frontend.
FHEM ist in **Perl** implementiert – dadurch sind Automatisierungen extrem flexibel, weil Logik sowohl über FHEM-Bordmittel (z. B. `notify`, `at`, `DOIF`) als auch über **Perl-Codeblöcke** möglich ist.

1.2 Zentrale Begriffe in FHEM

- **Device (Gerät):** Ein Objekt in FHEM, das eine reale Komponente (Lampe, Thermostat, Sensor) oder ein logisches Objekt (Dummy) repräsentiert.
- `define`**:** Legt ein Device an.
- `set`**:** Setzt einen Zustand / sendet einen Befehl (z. B. Licht ein/aus, Solltemperatur).
- `get`**:** Fragt Werte ab (z. B. Status, Messwert).
- **Readings:** Mess- oder Zustandswerte eines Devices (z. B. `temperature`, `humidity`, `state`). Readings sind die Basis für Automatisierungen.
- **Events:** Wenn sich Readings ändern oder ein `set` passiert, erzeugt FHEM Events. Diese Events können Regeln auslösen.

1.3 Automatisierung in FHEM

Automatisierungen basieren auf dem Event-Prinzip:

- `notify`**:** Reagiert auf Events eines Devices (z. B. „wenn Temperatur < 20°C, dann Heizung höher").

- **`at`:** Zeitgesteuerte Aktionen (z. B. täglich um 22:00 Licht aus).
- **`DOIF`:** Komfortabler Regelbaustein (wenn-dann-sonst), der komplexere Bedingungen lesbarer macht.
- **Perl in FHEM:** In vielen FHEM-Kommandos kann Perl direkt eingesetzt werden, z. B. in `{ ... }` Blöcken, um Berechnungen, Plausibilitätsprüfungen oder mehrstufige Abläufe umzusetzen.

1.4 Struktur eines Smart-Home-Setups (empfohlen)

Für ein sauberes System trennt man:

1. **Geräteanbindung** (z. B. MQTT-Devices, Funk-Thermostat-Modul, Wetterstation)
2. **Abstraktion / Logik** (z. B. „Wohnzimmer-Licht“, „Komfortmodus“)
3. **Automatisierungsregeln** (Zeitpläne, Reaktionen auf Sensorwerte)
4. **Sicherheit/Robustheit** (Default-Werte, Fehlerfälle, Logging)

---

2) Beispiel-Code in FHEM (Gerüst für Licht, Heizung, Wetterstation)

> **Wichtig:** Das ist ein „Template“, das du bei dir anpasst.
> Ich nutze unten **Dummy-Devices**, damit das Beispiel sofort verständlich ist. In der Praxis ersetzt du die Dummys durch echte Devices (z. B. MQTT-Device für Shelly, Zigbee-Lampe, Thermostat-Modul usw.).

2.1 Grundlegende Geräte (Licht, Heizung, Sensorik)

```
########################
# GRUND-GERÄTE (Demo)
########################

# Licht im Wohnzimmer (Demo)
define Licht_WZ dummy
attr Licht_WZ room Wohnzimmer
attr Licht_WZ setList on off toggle

# Heizung Wohnzimmer: Solltemperatur (Demo)
define Heizung_WZ dummy
attr Heizung_WZ room Wohnzimmer
attr Heizung_WZ setList desiredTemp:slider,16,0.5,26

# Wetterstation / Sensor: Temperatur + Luftfeuchte (Demo)
define Wetterstation dummy
attr Wetterstation room Sensorik

# Startwerte (optional)
setreading Wetterstation temperature 21.0
setreading Wetterstation humidity 45
```

So kannst du in der Theorie erklären:

- **Licht_WZ** repräsentiert das Licht, **`set on/off`** simuliert Schalten.
- **Heizung_WZ** besitzt ein Reading/Set-Wert **`desiredTemp`**.
- **Wetterstation** liefert Readings wie **`temperature`** und **`humidity`**.

2.2 Wetter/Temperatur: „Readings“ automatisch aktualisieren (Beispiel)

Wenn du reale Sensoren hast, kommen Readings meist über das jeweilige Modul. Für die Diplomarbeit ist es hilfreich zu zeigen, wie Readings verarbeitet werden.

Simulation: alle 60 Sekunden zufällig ändern (nur Demo):

```
define simWetterstation at +*00:01:00 {
  my $t = ReadingsNum("Wetterstation","temperature",21);
  my $h = ReadingsNum("Wetterstation","humidity",45);

  # kleine Änderung simulieren
  $t += (rand(0.6)-0.3);
  $h += (rand(2.0)-1.0);

  # Grenzen
  $t = 18 if $t < 18; $t = 28 if $t > 28;
  $h = 30 if $h < 30; $h = 70 if $h > 70;

  fhem("setreading Wetterstation temperature ".sprintf('%.1f',$t));
  fhem("setreading Wetterstation humidity ".int($h));
}
```

Theoriepunkt dazu:

- **`ReadingsNum()`** liest numerische Readings sicher aus.
- Automatisierung über **`at`** läuft periodisch (**`+*00:01:00`**).
- **`setreading`** schreibt Werte in FHEM, wodurch Events entstehen → Regeln können reagieren.

2.3 Lichtsteuerung: Zeitplan + Toggle + „Auto-Aus“

a) Zeitplan (z. B. abends an, nachts aus):

```
define LichtAbendsAn at *18:30:00 set Licht_WZ on
define LichtNachtsAus at *23:30:00 set Licht_WZ off
```

b) Auto-Aus 10 Minuten nach Einschalten (Event-basiert):

```
define n_LichtAutoAus notify Licht_WZ:on {
  # 10 Minuten nach ON wieder OFF
  fhem("define tmp_LichtAutoAus at +00:10:00 set Licht_WZ off");
  fhem("attr tmp_LichtAutoAus room Automatik");
}
```

Wichtig: Das ist eine einfache Demo. In einem „sauberen“ System würdest du vorher prüfen, ob **`tmp_LichtAutoAus`** schon existiert und ggf. ersetzen – kann man als Robustheitsaspekt beschreiben.

2.4 Heizung: Solltemperatur abhängig von Modus und Temperatur

Wir bauen eine typische Logik:

- **Modus**: **`Komfort`**, **`Eco`**, **`Abwesend`**
- Temperaturwerte kommen von **`Wetterstation:temperature`**
- Heizung bekommt eine **Solltemperatur** (**`set Heizung_WZ desiredTemp X`**)

a) Modus-Device:

```
define Modus dummy
attr Modus room Automatik
attr Modus setList Komfort Eco Abwesend
set Modus Komfort
```

b) Regel: abhängig vom Modus Solltemperatur setzen

```
define n_HeizungModus notify Modus:.* {
  my $m = ReadingsVal("Modus","state","Komfort");

  my %ziel = (
    "Komfort"  => 22.0,
    "Eco"      => 19.0,
    "Abwesend" => 16.5
  );

  my $sp = $ziel{$m};
  fhem("set Heizung_WZ desiredTemp $sp");
}
```

c) Zusatzregel: wenn Raumtemperatur deutlich unter Soll → „Heizbedarf“ setzen (logisch)
Damit kannst du in der Arbeit zeigen, wie Regelung/Entscheidung aussehen kann. (Ob du dann wirklich ein Relais/Heizventil schaltest, hängt vom System ab.)

```
define Heizbedarf dummy
attr Heizbedarf room Automatik
attr Heizbedarf setList on off
set Heizbedarf off

define n_Heizbedarf notify Wetterstation:temperature {
  my $ist = ReadingsNum("Wetterstation","temperature",21);
  my $soll = ReadingsNum("Heizung_WZ","desiredTemp",22);

  # Hysterese: verhindert ständiges Ein/Aus
  if ($ist < $soll - 0.3) {
    fhem("set Heizbedarf on");
  } elsif ($ist > $soll + 0.3) {
    fhem("set Heizbedarf off");
  }
}
```

Theorie, die du dazu schreiben kannst:

- **Hysterese** vermeidet häufiges Schalten (Schaltflattern).
- Trennung von „Sollwert“ (Thermostat) und „Heizbedarf“ (Entscheidungsvariable).
- Ereignisgesteuerte Auswertung bei jedem neuen Temperaturwert.

2.5 Wetter-Forecast einbinden (echte Außen-/Online-Wetterdaten)

Wenn du statt einer eigenen Wetterstation auch Online-Wetter verwenden willst, gibt es in FHEM typischerweise Weather-Module (abhängig vom Setup/Provider). In der Diplomarbeit reicht oft:

- Außenwerte als zusätzlicher Input (z. B. zur Heizstrategie, Fenster-Offen-Erkennung, Lüften, Frostschutz)
- Datenquelle (API/Provider) → Readings → Automatisierung

Praxis-Tipp für deine Arbeit:
Du kannst es so formulieren: *„Die Wetterdaten werden durch ein FHEM-Weather-Device zyklisch aktualisiert und als Readings (Temperatur, Luftdruck, Prognose) bereitgestellt. Diese Readings können wie Sensordaten lokaler Hardware in Regeln verwendet werden.“*

---

3) So ersetzt du die Dummys durch echte Geräte (kurz & diplomarbeits-tauglich)

- **Licht:** Dummy ersetzen durch z. B. MQTT-Device (Shelly/Zigbee2MQTT), Z-Wave, KNX etc.
  - Dann bleibt deine Logik gleich: `set <Device> on/off`.
- **Heizung:** Dummy ersetzen durch Thermostat-Device (z. B. Homematic, MAX!, tado via Bridge, MQTT-Thermostat).
  - Dann bleibt gleich: `set <Thermostat> desiredTemp <wert>` (je nach Modul heißt es evtl. `desiredTemperature`, `setpoint`, …).
- **Wetterstation/Sensor:** Dummy ersetzen durch reales Sensor-Device (ESPHome via MQTT, Arduino → MQTT/HTTP, Zigbee-Sensor).
  - Wichtig ist: Readings heißen dann z. B. `temperature`, `humidity` – oder du mapst sie um.

---

4) Wenn du willst: „echter" Code passend zu deinem Setup

Damit ich dir **den finalen, wirklich lauffähigen Code** für *dein* Projekt schreiben kann (ohne Dummys), sag mir kurz:

1. Wie sind deine Geräte angebunden? (MQTT? Zigbee2MQTT? KNX? Homematic? Shelly? Arduino → seriell/HTTP/MQTT?)
2. Welche Namen/Readings liefern deine Sensoren aktuell in FHEM? (z. B. `temperature`, `temp`, `T`, …)

Bis dahin kannst du das obige Gerüst 1:1 als **Kapitel „Konzept + Implementierung"** verwenden und danach die Dummys durch echte Devices ersetzen.

> Gib mir den Ganzen code dazu und bitte nicht mit Dummy Daten

> Wir machen es mit einen Rasbeery pi 4
> , einen Arduino, mit MQTT und mit NODE RED

Alles klar — dann machen wir es **realistisch mit Raspberry Pi 4 + Mosquitto (MQTT) + FHEM + Node-RED + Arduino** und **ohne Dummy-Devices**.

Du bekommst hier:

1. **Arduino-Code** (MQTT Client) für

- **Licht (Relais)**

- **Heizung (Relais / Ventil) mit Solltemperatur**
- **Wetterstation** (Temperatur + Feuchte; optional Druck)

2. **FHEM kompletter Code** (MQTT2_SERVER + MQTT2_DEVICE + Readings + setList + Automatikregeln)
3. **Node-RED Flow** (Dashboard/Steuerung + MQTT Brücke)

   Wichtig (realistisch): Ein "Arduino UNO" kann **nicht** direkt per WLAN MQTT sprechen. Du brauchst entweder

   - **ESP32 / ESP8266** (am einfachsten), oder
   - Arduino UNO + **Ethernet Shield (W5500/W5100)**.
     Damit du sofort weiterkommst, gebe ich dir **ESP32-Version (empfohlen)** + **UNO+Ethernet-Variante**.

---

0) MQTT-Topic-Standard (für alles identisch)

Wir verwenden saubere Topics (damit FHEM/Node-RED easy bleibt):

Commands (Pi → Arduino):

- `da/wohnzimmer/licht/cmd` payload: `ON` / `OFF` / `TOGGLE`
- `da/wohnzimmer/heizung/cmd` payload: `ON` / `OFF`
- `da/wohnzimmer/heizung/setpoint` payload: z.B. `22.0`

States (Arduino → Pi):

- `da/wohnzimmer/licht/state` payload: `ON` / `OFF`
- `da/wohnzimmer/heizung/state` payload: `ON` / `OFF`
- `da/wetter/temperature` payload: z.B. `21.7`
- `da/wetter/humidity` payload: z.B. `44.2`
- (optional) `da/wetter/pressure` payload: z.B. `1012.8`

---

1) Arduino Code (ESP32) – Licht + Heizung + Wetterstation (DHT22 + optional BMP280)

Hardware-Beispiel:

- ESP32
- Relais Licht an GPIO `26`
- Relais Heizung an GPIO `27`
- DHT22 an GPIO `4`
- optional BMP280 via I2C (SDA 21, SCL 22)

  Passe die Pins oben in den Defines an, wenn du andere hast.

```
/*
  ESP32 MQTT SmartHome Node
  - Licht: Relais
  - Heizung: Relais + Solltemperatur (Hysterese)
  - Wetterstation: DHT22 (Temp/Humidity) + optional BMP280 (Pressure)

  Topics:
    da/wohnzimmer/licht/cmd         ON|OFF|TOGGLE
    da/wohnzimmer/heizung/cmd       ON|OFF
    da/wohnzimmer/heizung/setpoint float (e.g. 22.0)

    da/wohnzimmer/licht/state       ON|OFF
    da/wohnzimmer/heizung/state     ON|OFF
    da/wetter/temperature           float
    da/wetter/humidity              float
    da/wetter/pressure              float (optional)
*/

#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

// Optional BMP280:
#define USE_BMP280 0
#if USE_BMP280
  #include <Wire.h>
  #include <Adafruit_BMP280.h>
  Adafruit_BMP280 bmp;
#endif

// ======== CONFIG ========
const char* WIFI_SSID = "DEIN_WLAN";
const char* WIFI_PASS = "DEIN_PASSWORT";

const char* MQTT_HOST = "192.168.1.10";   // IP vom Raspberry Pi (Mosquitto)
const uint16_t MQTT_PORT = 1883;

// Relais Pins (anpassen!)
const int PIN_RELAY_LIGHT = 26;
const int PIN_RELAY_HEAT  = 27;

// DHT Sensor
#define DHTPIN 4
#define DHTTYPE DHT22

// Hysterese für Heizung (anpassen)
float HYST = 0.3;

// Publish-Interval
unsigned long SENSOR_INTERVAL_MS = 10000; // alle 10s

// ======== TOPICS ========
const char* T_LIGHT_CMD     = "da/wohnzimmer/licht/cmd";
const char* T_LIGHT_STATE   = "da/wohnzimmer/licht/state";

const char* T_HEAT_CMD      = "da/wohnzimmer/heizung/cmd";
const char* T_HEAT_SETPOINT = "da/wohnzimmer/heizung/setpoint";
const char* T_HEAT_STATE    = "da/wohnzimmer/heizung/state";

const char* T_TEMP          = "da/wetter/temperature";
const char* T_HUM           = "da/wetter/humidity";
const char* T_PRESS         = "da/wetter/pressure";

// ======== RUNTIME ========
WiFiClient espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);

bool lightOn = false;
bool heatOn  = false;

float setpoint = 22.0;
float lastTemp = NAN;
float lastHum  = NAN;
```

```
unsigned long lastSensorTs = 0;

static const char* onOff(bool v) { return v ? "ON" : "OFF"; }

void applyRelays() {
  digitalWrite(PIN_RELAY_LIGHT, lightOn ? HIGH : LOW);
  digitalWrite(PIN_RELAY_HEAT,  heatOn  ? HIGH : LOW);
}

void publishStates(bool force=false) {
  static String lastLight, lastHeat;
  String l = onOff(lightOn);
  String h = onOff(heatOn);

  if(force || l != lastLight) {
    mqtt.publish(T_LIGHT_STATE, l.c_str(), true);
    lastLight = l;
  }
  if(force || h != lastHeat) {
    mqtt.publish(T_HEAT_STATE, h.c_str(), true);
    lastHeat = h;
  }
}

void setLight(bool v) {
  lightOn = v;
  applyRelays();
  publishStates();
}

void toggleLight() { setLight(!lightOn); }

void setHeat(bool v) {
  heatOn = v;
  applyRelays();
  publishStates();
}

bool parseFloat(const char* s, float &out) {
  if(!s || !*s) return false;
  char* endp = nullptr;
  out = strtof(s, &endp);
  return endp && endp != s;
}

void controlHeatingWithTemp(float t) {
  // simple thermostat logic with hysteresis around setpoint
  if (isnan(t)) return;

  if (!heatOn && t < (setpoint - HYST)) {
    setHeat(true);
  } else if (heatOn && t > (setpoint + HYST)) {
    setHeat(false);
  }
}

void mqttCallback(char* topic, byte* payload, unsigned int len) {
  payload[len] = '\0';
  String msg((char*)payload);

  if (strcmp(topic, T_LIGHT_CMD) == 0) {
    msg.trim();
    msg.toUpperCase();
    if (msg == "ON") setLight(true);
    else if (msg == "OFF") setLight(false);
    else if (msg == "TOGGLE") toggleLight();
  }

  if (strcmp(topic, T_HEAT_CMD) == 0) {
    msg.trim();
    msg.toUpperCase();
    if (msg == "ON") setHeat(true);
    else if (msg == "OFF") setHeat(false);
  }
```

```
  if (strcmp(topic, T_HEAT_SETPOINT) == 0) {
    float sp;
    if (parseFloat(msg.c_str(), sp)) {
      if (sp < 10) sp = 10;
      if (sp > 30) sp = 30;
      setpoint = sp;
      // nach Setpoint-Änderung sofort nachregeln
      controlHeatingWithTemp(lastTemp);
    }
  }
}

void ensureWifi() {
  if (WiFi.status() == WL_CONNECTED) return;
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void ensureMqtt() {
  if (mqtt.connected()) return;

  while (!mqtt.connected()) {
    String clientId = "DA-ESP32-" + String((uint32_t)ESP.getEfuseMac(), HEX);
    if (mqtt.connect(clientId.c_str())) {
      mqtt.subscribe(T_LIGHT_CMD);
      mqtt.subscribe(T_HEAT_CMD);
      mqtt.subscribe(T_HEAT_SETPOINT);
      publishStates(true);
    } else {
      delay(1500);
    }
  }
}

void publishSensors(float t, float h) {
  char buf[16];

  if (!isnan(t)) {
    dtostrf(t, 0, 1, buf);
    mqtt.publish(T_TEMP, buf, true);
  }
  if (!isnan(h)) {
    dtostrf(h, 0, 1, buf);
    mqtt.publish(T_HUM, buf, true);
  }

#if USE_BMP280
  float p = bmp.readPressure() / 100.0f; // hPa
  if (!isnan(p)) {
    dtostrf(p, 0, 1, buf);
    mqtt.publish(T_PRESS, buf, true);
  }
#endif
}

void setup() {
  pinMode(PIN_RELAY_LIGHT, OUTPUT);
  pinMode(PIN_RELAY_HEAT, OUTPUT);
  digitalWrite(PIN_RELAY_LIGHT, LOW);
  digitalWrite(PIN_RELAY_HEAT, LOW);

  dht.begin();

#if USE_BMP280
  Wire.begin();
  bmp.begin(0x76); // ggf. 0x77
#endif

  ensureWifi();
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(mqttCallback);
  ensureMqtt();
```

```
  ensureMqtt();
}

void loop() {
  ensureWifi();
  ensureMqtt();
  mqtt.loop();

  unsigned long now = millis();
  if (now - lastSensorTs >= SENSOR_INTERVAL_MS) {
    lastSensorTs = now;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    // Nur wenn valide
    if (!isnan(t)) lastTemp = t;
    if (!isnan(h)) lastHum  = h;

    publishSensors(lastTemp, lastHum);
    controlHeatingWithTemp(lastTemp);
  }
}
```

---

2) Arduino UNO + Ethernet Shield (W5500/W5100) – Variante (falls ihr wirklich UNO nutzt)

Wenn ihr wirklich UNO nutzt, dann **Ethernet Shield** und statt `WiFi.h` nutzt du `Ethernet.h`. Das ist technisch länger – aber das Muster ist dasselbe.
Wenn du willst, kann ich dir den UNO+W5500 Sketch 1:1 fertig ausgeben, aber der ESP32 ist für Diplomarbeit + MQTT/IoT in der Praxis deutlich besser.

---

3) Raspberry Pi 4 – Mosquitto installieren (kurz)

Auf dem Pi:

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto
```

Test:

```
mosquitto_sub -t 'da/#'
```

---

4) FHEM kompletter Code (MQTT Geräte + Readings + Steuerung + Automatik)

4.1 MQTT Server in FHEM

In FHEM (oder in fhem.cfg):

```
########################################
# MQTT Broker (Mosquitto) läuft am Pi
########################################
define MQTTBroker MQTT2_SERVER 1883
attr MQTTBroker room MQTT
```

4.2 MQTT Device für Arduino/ESP32 (Topics in Readings abbilden)

```
########################################
# MQTT2_DEVICE: liest alle Topics da/#
########################################
define DA_MQTT MQTT2_DEVICE
attr DA_MQTT room MQTT
attr DA_MQTT subscribeReading_da_wetter_temperature  da/wetter/temperature
attr DA_MQTT subscribeReading_da_wetter_humidity     da/wetter/humidity
attr DA_MQTT subscribeReading_da_wetter_pressure     da/wetter/pressure

attr DA_MQTT subscribeReading_licht_state            da/wohnzimmer/licht/state
attr DA_MQTT subscribeReading_heizung_state          da/wohnzimmer/heizung/state

# Hilfreich fürs Debuggen:
attr DA_MQTT verbose 3
```

Damit bekommst du in FHEM echte Readings wie:

- **`da_wetter_temperature`**
- **`da_wetter_humidity`**
- **`licht_state`**
- **`heizung_state`**

4.3 „Virtuelle Schalter“ in FHEM, die wirklich per MQTT schalten (kein Dummy!)

Wir machen **FHEM-Devices vom Typ `MQTT2_DEVICE` als Sender** über **`setList`/`set`** → **`publish`**.
In FHEM ist das sauber, indem du einfach **`set <MQTT2_DEVICE> publish <topic> <payload>`** nutzt.

Ich definiere dir dafür **einen Satz FHEM-Kommandos** (Licht / Heizung / Setpoint):

```
########################################
# Licht Kommandos
########################################
define Licht_WZ notify global:INITIALIZED {
  # nur Platzhalter um im Raum "Wohnzimmer" aufzutauchen (kein Dummy)
}
attr Licht_WZ room Wohnzimmer
attr Licht_WZ event-on-change-reading .*

# WebCmd Buttons (optional)
attr Licht_WZ webCmd on:off:toggle

# set-Konfiguration über userReadings + notify ist möglich,
# aber am stabilsten ist es über Befehle (siehe unten) im WebCmd.

define cmd_LichtWZ_on  notify Licht_WZ:.* { fhem("set MQTTBroker publish da/wohnzimmer/licht/cmd ON")
define cmd_LichtWZ_off notify Licht_WZ:.* { fhem("set MQTTBroker publish da/wohnzimmer/licht/cmd OFF")
define cmd_LichtWZ_tog notify Licht_WZ:.* { fhem("set MQTTBroker publish da/wohnzimmer/licht/cmd TOGGL

# Damit die 3 notifies nicht immer feuern, stellen wir es sauber über separate Befehle:
delete cmd_LichtWZ_on
delete cmd_LichtWZ_off
delete cmd_LichtWZ_tog
```

Oben hast du gesehen: So geht’s, aber schöner ist, wir machen es **sauber als eigenes FHEM-Device mit setList**, das direkt publish macht.

✅ Saubere Variante: eigenes Perl-Modul als Command-Device (ohne Dummy)

Dafür legst du eine Datei in FHEM an:

/opt/fhem/FHEM/99_DA_SmartHome.pm

```
package main;
use strict;
use warnings;

sub DA_SmartHome_Initialize($) {
  my ($hash) = @_;
  $hash->{DefFn}    = "DA_SmartHome_Define";
  $hash->{SetFn}    = "DA_SmartHome_Set";
  $hash->{AttrList} = "room mqttDevice " . $readingFnAttributes;
}

sub DA_SmartHome_Define($$) {
  my ($hash, $def) = @_;
  my @a = split("[ \t]+", $def);

  return "Usage: define <name> DA_SmartHome <mqtt2_server_name>"
    if (@a != 3);

  $hash->{MQTTDEV} = $a[2];
  return undef;
}

sub _pub {
  my ($mqttDev, $topic, $payload) = @_;
  fhem("set $mqttDev publish $topic $payload");
}

sub DA_SmartHome_Set($@) {
  my ($hash, @a) = @_;
  my $name = $a[0];
  my $cmd  = $a[1] // "";

  my $mqttDev = AttrVal($name, "mqttDevice", $hash->{MQTTDEV});
  return "mqttDevice not set or invalid" if (!$mqttDev);

  my $list = join(" ",
    "light:on,off,toggle",
    "heat:on,off",
    "setpoint:slider,10,0.5,30"
  );

  return "Unknown argument $cmd, choose one of $list" if ($cmd eq "" || $cmd eq "?");

  if ($cmd eq "light") {
    my $v = uc($a[2] // "");
    return "Usage: set $name light on|off|toggle" if ($v !~ /^(ON|OFF|TOGGLE)$/);
    _pub($mqttDev, "da/wohnzimmer/licht/cmd", $v);
    return undef;
  }

  if ($cmd eq "heat") {
    my $v = uc($a[2] // "");
    return "Usage: set $name heat on|off" if ($v !~ /^(ON|OFF)$/);
    _pub($mqttDev, "da/wohnzimmer/heizung/cmd", $v);
    return undef;
  }

  if ($cmd eq "setpoint") {
    my $sp = $a[2];
    return "Usage: set $name setpoint <float>" if (!defined $sp || $sp !~ /^-?\d+(\.\d+)?$/);
    $sp = 10 if $sp < 10;
    $sp = 30 if $sp > 30;
    _pub($mqttDev, "da/wohnzimmer/heizung/setpoint", $sp);
    return undef;
  }

  return "Unknown argument $cmd, choose one of $list";
}

1;
```

Dann in FHEM:

```
########################################
# Dein SmartHome Command Device
########################################
define DA_Control DA_SmartHome MQTTBroker
attr DA_Control room Wohnzimmer
attr DA_Control mqttDevice MQTTBroker
attr DA_Control webCmd light:on:light:off:light:toggle:setpoint:heat:on:heat:off
```

Jetzt kannst du wirklich in FHEM klicken/steuern:

- **`set DA_Control light on`**
- **`set DA_Control setpoint 22.0`**
- **`set DA_Control heat off`**

und das geht **real** per MQTT zum Arduino.

4.4 Automatik in FHEM (optional zusätzlich zur Arduino-Regelung)

Du hast schon im Arduino eine Thermostatlogik. In der Diplomarbeit ist es oft gut zu zeigen, dass man Logik auch zentral machen kann.

Beispiel: Wenn Temperatur unter 18°C → Setpoint hoch auf 22 (Komfort), sonst Eco:

```
define n_tempAuto notify DA_MQTT:da_wetter_temperature:.* {
  my $t = ReadingsNum("DA_MQTT","da_wetter_temperature",0);

  if ($t < 18) {
    fhem("set DA_Control setpoint 22.0");
  } elsif ($t > 22) {
    fhem("set DA_Control setpoint 19.0");
  }
}
attr n_tempAuto room Automatik
```

---

5) Node-RED Flow (Dashboard + MQTT)

Das ist ein **fertiger Import-Flow** (Menu → Import → Clipboard).

Er macht:

- Buttons Licht ON/OFF/TOGGLE
- Slider Setpoint
- Anzeige von Temperatur/Feuchte (aus MQTT)

  Vorher in Node-RED MQTT-Broker auf **`localhost:1883`** einstellen.

```
[
  {
    "id": "tab_da",
    "type": "tab",
    "label": "DA SmartHome",
    "disabled": false,
    "info": ""
  },
  {
    "id": "mqtt_local",
    "type": "mqtt-broker",
    "name": "Mosquitto (local)",
    "broker": "127.0.0.1",
    "port": "1883",
    "clientid": "nodered-da",
    "usetls": false,
    "protocolVersion": "4",
    "keepalive": "60",
    "cleansession": true,
    "autoConnect": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthPayload": "",
    "closeTopic": "",
    "closeQos": "0",
    "closePayload": "",
    "willTopic": "",
    "willQos": "0",
    "willPayload": ""
  },
  {
    "id": "ui_tab_da",
    "type": "ui_tab",
    "name": "SmartHome",
    "icon": "dashboard",
    "order": 1
  },
  {
    "id": "ui_group_ctrl",
    "type": "ui_group",
    "name": "Wohnzimmer Steuerung",
    "tab": "ui_tab_da",
    "order": 1,
    "disp": true,
    "width": "8",
    "collapse": false
  },
  {
    "id": "ui_group_sens",
    "type": "ui_group",
    "name": "Sensoren",
    "tab": "ui_tab_da",
    "order": 2,
    "disp": true,
    "width": "8",
    "collapse": false
  },
  {
    "id": "btn_light_on",
    "type": "ui_button",
    "z": "tab_da",
    "name": "Licht ON",
    "group": "ui_group_ctrl",
    "order": 1,
    "width": 2,
    "height": 1,
    "passthru": false,
    "label": "Licht ON",
    "payload": "ON",
    "payloadType": "str",
    "topic": "da/wohnzimmer/licht/cmd",
    "x": 140,
    "y": 80,
    "wires": [["mqtt_out_light"]]
  },
```

```
  {
    "id": "btn_light_off",
    "type": "ui_button",
    "z": "tab_da",
    "name": "Licht OFF",
    "group": "ui_group_ctrl",
    "order": 2,
    "width": 2,
    "height": 1,
    "passthru": false,
    "label": "Licht OFF",
    "payload": "OFF",
    "payloadType": "str",
    "topic": "da/wohnzimmer/licht/cmd",
    "x": 140,
    "y": 120,
    "wires": [["mqtt_out_light"]]
  },
  {
    "id": "btn_light_toggle",
    "type": "ui_button",
    "z": "tab_da",
    "name": "Licht TOGGLE",
    "group": "ui_group_ctrl",
    "order": 3,
    "width": 2,
    "height": 1,
    "passthru": false,
    "label": "Licht TOGGLE",
    "payload": "TOGGLE",
    "payloadType": "str",
    "topic": "da/wohnzimmer/licht/cmd",
    "x": 150,
    "y": 160,
    "wires": [["mqtt_out_light"]]
  },
  {
    "id": "slider_sp",
    "type": "ui_slider",
    "z": "tab_da",
    "name": "Solltemp",
    "label": "Solltemp (°C)",
    "group": "ui_group_ctrl",
    "order": 4,
    "width": 6,
    "height": 1,
    "passthru": true,
    "outs": "all",
    "topic": "da/wohnzimmer/heizung/setpoint",
    "min": 10,
    "max": 30,
    "step": 0.5,
    "x": 150,
    "y": 220,
    "wires": [["mqtt_out_sp"]]
  },
  {
    "id": "mqtt_out_light",
    "type": "mqtt out",
    "z": "tab_da",
    "name": "",
    "topic": "",
    "qos": "1",
    "retain": "true",
    "broker": "mqtt_local",
    "x": 420,
    "y": 120,
    "wires": []
  },
  {
    "id": "mqtt_out_sp",
    "type": "mqtt out",
    "z": "tab_da",
    "name": "",
    "topic": "",
```

```
    "qos": "1",
    "retain": "true",
    "broker": "mqtt_local",
    "x": 420,
    "y": 220,
    "wires": []
  },
  {
    "id": "mqtt_in_temp",
    "type": "mqtt in",
    "z": "tab_da",
    "name": "Temp",
    "topic": "da/wetter/temperature",
    "qos": "1",
    "datatype": "auto",
    "broker": "mqtt_local",
    "nl": false,
    "rap": true,
    "rh": 0,
    "x": 110,
    "y": 320,
    "wires": [["ui_temp"]]
  },
  {
    "id": "mqtt_in_hum",
    "type": "mqtt in",
    "z": "tab_da",
    "name": "Hum",
    "topic": "da/wetter/humidity",
    "qos": "1",
    "datatype": "auto",
    "broker": "mqtt_local",
    "nl": false,
    "rap": true,
    "rh": 0,
    "x": 110,
    "y": 360,
    "wires": [["ui_hum"]]
  },
  {
    "id": "ui_temp",
    "type": "ui_text",
    "z": "tab_da",
    "group": "ui_group_sens",
    "order": 1,
    "width": 6,
    "height": 1,
    "name": "",
    "label": "Temperatur",
    "format": "{{msg.payload}} °C",
    "layout": "row-spread",
    "x": 340,
    "y": 320,
    "wires": []
  },
  {
    "id": "ui_hum",
    "type": "ui_text",
    "z": "tab_da",
    "group": "ui_group_sens",
    "order": 2,
    "width": 6,
    "height": 1,
    "name": "",
    "label": "Luftfeuchte",
    "format": "{{msg.payload}} %",
    "layout": "row-spread",
    "x": 340,
    "y": 360,
    "wires": []
  }
]
```

6) Was du jetzt konkret nur noch anpassen musst

- **ESP32 Sketch:** `WIFI_SSID`, `WIFI_PASS`, `MQTT_HOST` (Pi IP)
- **Pins:** `PIN_RELAY_LIGHT`, `PIN_RELAY_HEAT`, `DHTPIN`
- **FHEM:** Datei `99_DA_SmartHome.pm` nach `/opt/fhem/FHEM/` kopieren und FHEM neu laden:
  - in FHEM: `shutdown restart`
  - oder `reload 99_DA_SmartHome.pm` (wenn möglich)

Wenn du mir sagst, **welches Board ihr wirklich am Arduino habt (ESP32/ESP8266 oder UNO+Ethernet)** und **welche Sensoren genau** (DHT22/BME280/BMP280/DS18B20), kann ich dir den Arduino-Teil **100% passend** (inkl. Libraries und korrekter Verdrahtung) final „glatt“ machen — aber der oben ist bereits ein vollständiger, realer MQTT→FHEM/Node-RED Smart-Home Stack ohne Dummy-Daten.